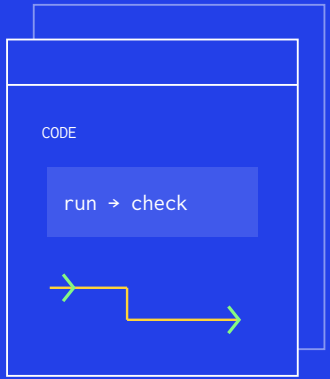
WEEK 3 / LAB 6 / TEAM WORK

MLOps: Machine Learning Operations

Make the pipeline able to fail.

Run your checks in GitHub Actions, write tests that catch defects, watch the pipeline fail and recover, then deploy to Cloud Run. Needs Lab 5 merged on main.



By the end you can show three things

Each one must be visible in your team repository.

1. **Cover**

   Every change going into your integration branches is checked on a clean machine, the checks cover what matters here, and a change that passes on main reaches your Cloud Run service.

2. **Catch**

   Your tests fail for the defects they claim to catch, and you can show it. One test recomputes a result you recorded earlier.

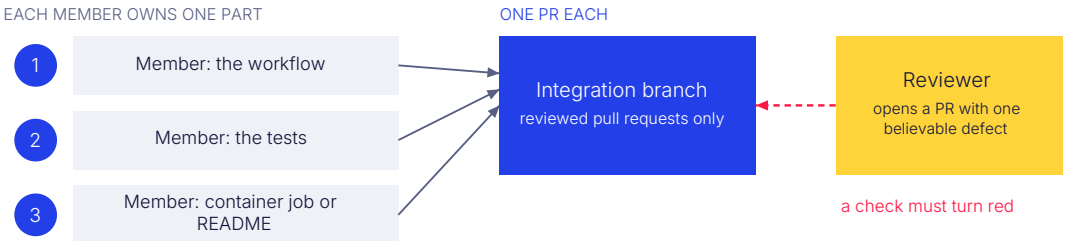
3. **Repair**

   You saw the pipeline fail for a real reason, found the cause in its logs, and fixed it.

Who does what in your team

The split is yours; every member owns something and reviews something.

The whole team agrees which checks must be green to merge, and how tolerances are set



Individual report reports/week-3.md covers Labs 5–6 in your personal repository.

What your team must add

Everything goes in through reviewed pull requests.

REQUIRED

Workflows only run from .github/workflows/ at the repository root. If you add a pytest marker, register it in pyproject.toml, or collection fails.



README: what runs, why, and how to reproduce a failure locally.

Three tools you will use

Lecture 6 explained the ideas. This is what you write or read in this lab.



Any tool that gives the same evidence is fine; the evidence matters, not the tool.

Questions your README must answer (1 of 3)

Five questions about what your pipeline checks.

README QUESTIONS

Answer each question in your README: your choice, the option you rejected, and why. Several answers can be good.

ANSWER EACH IN YOUR README

coverage	what do your checks not cover yet?
machine	what does a clean runner not have?
trigger	which changes should start which checks?
order	which check should fail first, and why?
evidence	what can CI claim about the model?

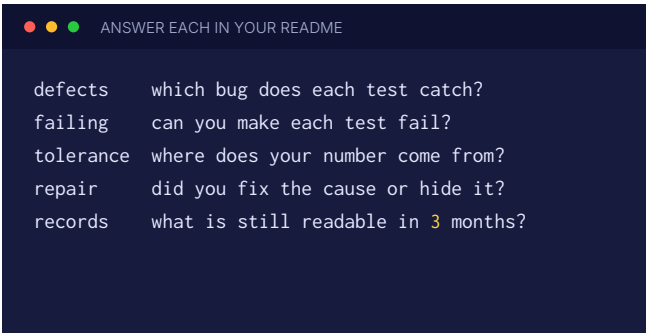
Examples and cheat sheets that follow are tools, not answers.

Questions your README must answer (2 of 3)

Five questions about whether your tests work.

README QUESTIONS

A test is worth what it can catch: name the defect, then prove the test can fail.



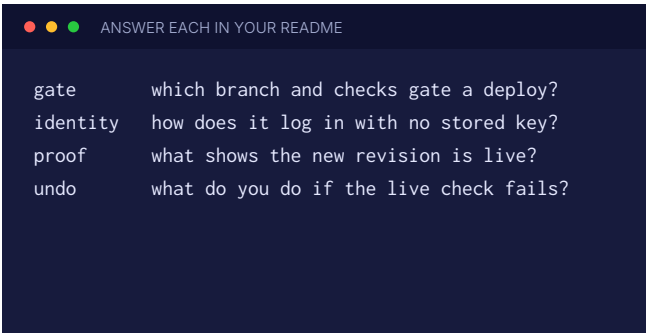
Reviewers read your reasons; there is no single correct answer.

Questions your README must answer (3 of 3)

Four questions about delivering to Cloud Run.

README QUESTIONS

Same format: your choice, the option you rejected, and why. Use the Cloud Run service and the identity check from Lab 5.



Use the project and permissions your instructor gives your team.

Rules that keep CI honest

Fixed for every team; everything else is your choice.

1 **Locked installs only**

Install from the lock file; never regenerate it just to get green. Keep the starter's safe setup: least privilege, no secrets, pinned versions, time limits.

2 **Never mute a failure**

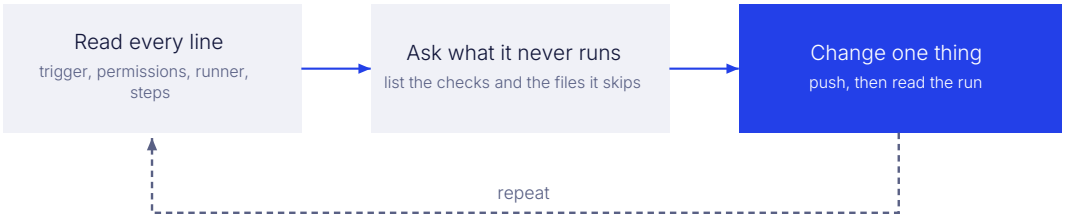
No ignored failures, skipped tests or loosened assertions to get green. The failure you show must be real: a change caused it and a check caught it.

3 **Keep evidence honest**

CI evidence is what the runner itself produced, not your laptop's MLflow runs. No test-period metrics. A tolerance comes from measured variation.

Start from the workflow you already have

Your starter repository already runs checks; read it before you add to it.



The supplied workflow is a starting point, not an answer.

Words to search in the Actions docs

Each word is a lever; decide which ones your pipeline needs.

KEYWORDS

Read the reference for each word. Copy nothing you cannot explain, and say in the README why you chose it.

WORKFLOW VOCABULARY

```
on: pull_request, push, branches
jobs, needs
steps: uses, run
permissions, timeout-minutes
concurrency, artifacts: upload, download
environment, if: main only
id-token: keyless login
```

Pin what you run: a tag can move, a commit SHA cannot.

Cheat sheet: pytest markers and selection

Select what runs, list what would run, stop early.

SELECT TESTS

Markers group tests. With --strict-markers an unregistered marker is an error, and pytest exits with code 5 when nothing is selected.



-m selects by marker, --collect-only lists what would run, -x stops at the first failure.

Cheat sheet: read a run from the terminal

The Checks tab shows the same logs; the command line is optional.

READ A RUN

Find the failing run, read only the failed steps, and save the text. Authenticate gh once; never paste a token into a report.

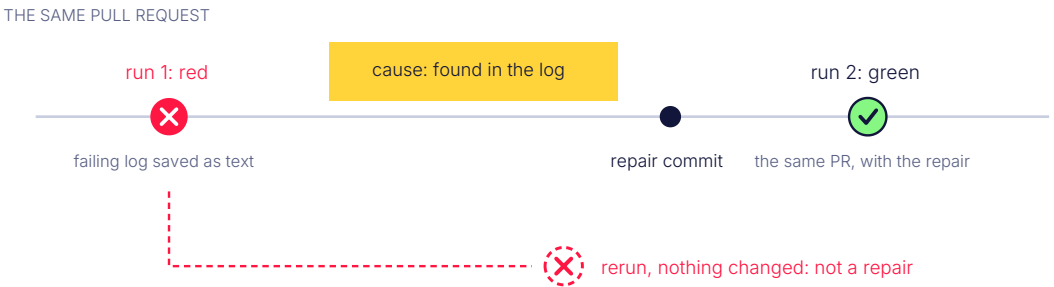
gh (optional)

```
$ gh run list --limit 5
$ gh run view RUN_ID --log-failed
$ gh run view RUN_ID --log
$ gh pr checks
```

Save the log as text in your repo or report; links alone do not last.

Red to green, with evidence

One failing run, one diagnosis, one repair, on the same PR.



Keep the failing log as text: run records expire, links alone do not last.

Break it on purpose

A teammate adds one believable defect in a PR; the author learns what the tests miss.



The reviewer picks the bug; the author must not know it in advance.

What CI can and cannot claim

Evidence belongs to the machine that produced it.

CI CAN CLAIM

runner-made facts

- ✓ this commit
- ✓ installed from the lock
- ✓ passed these checks

CI CANNOT CLAIM

laptop facts

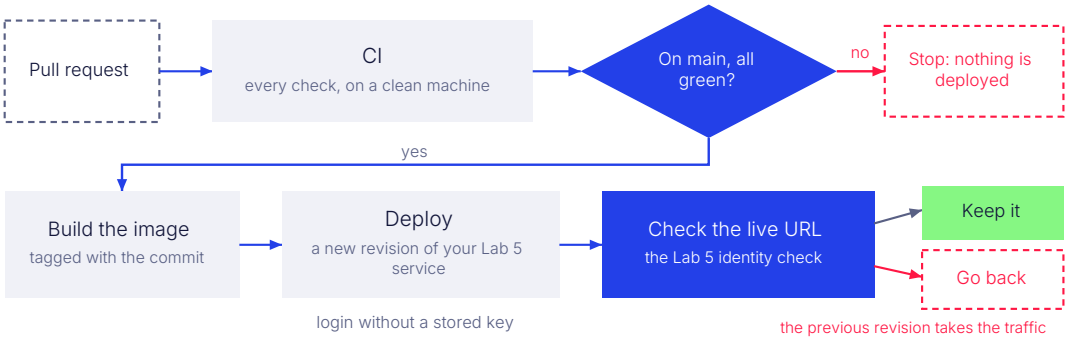
- ✗ your MLflow runs
- ✗ local model files
- ✗ anything the runner did not produce

README: three honest lines on what CI does and does not show about the model

Local runs and CI runs are separate claims; cite which one you mean.

Deliver: from a green run to Cloud Run

Same service and same identity check as Lab 5; now a workflow does the deploy.

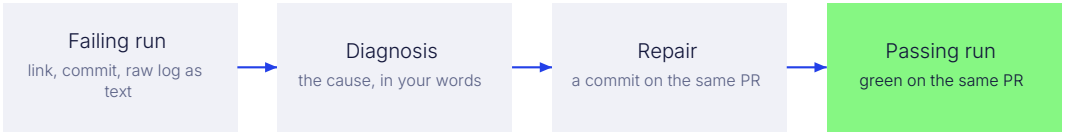


Keep the deploy run and the live check output, with the revision it checked.

Evidence to keep

What a reviewer will look for in your report and repository.

RED TO GREEN



AND THE REST

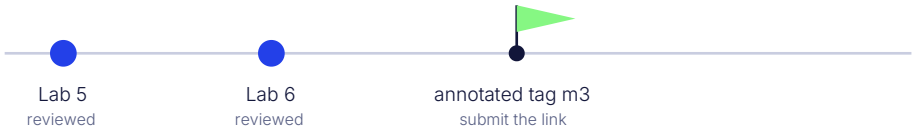
Test map
defect, test, and how you know it can fail

Deploy record
the deploy run and the live check output, with the revision it checked

m3 and your individual report

One team checkpoint for Labs 5–6; one personal report.

TEAM



YOU

reports/week-3.md

your own work, one decision, one blocker, any AI help

Sunday 23:59

10% of the course

Incomplete work is reported as incomplete; never invent a result or a passing check.

EXIT CHECK

Could your pipeline have failed? Show me.

Next, Lecture 7 and Lab 7: turn training into a pipeline with a gate.

Where the examples come from

Condensed from public documentation; read the originals for every option.

REFERENCES

Commands and keywords are condensed for this lab. Check the docs when an option is unclear.

SOURCES

```
docs.github.com  GitHub Actions: workflows
docs.pytest.org  markers and exit codes
docs.astral.sh/uv  uv in GitHub Actions
cli.github.com/manual  gh run, gh pr checks
cloud.google.com/run  deploying and revisions
```

Illustrative examples; run them in your own project before you rely on them.